

Comunicação MPI

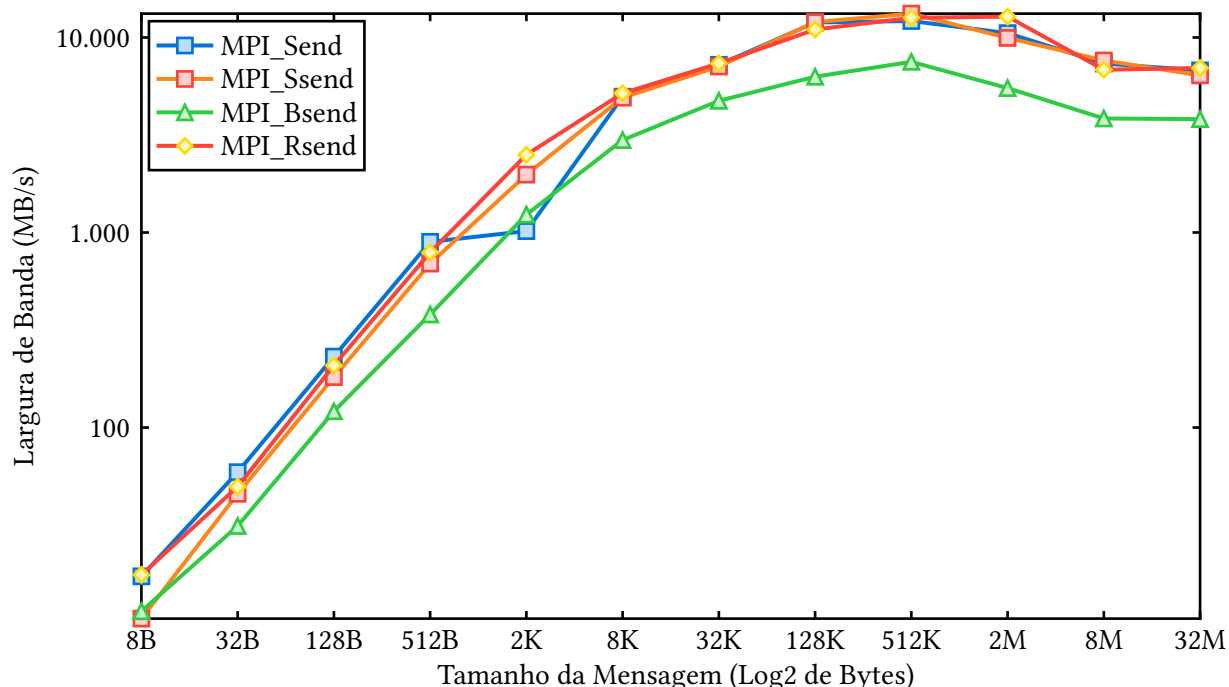
I. Metodologia

O experimento consistiu em utilizar um programa que para cada tipo de comunicação, são executadas cem iterações de comunicação para cada tamanho de carga, que varia de 8 Bytes até 32MB.

Nesse contexto, foram testados os seguintes métodos de comunicação:

- MPI_Send
- MPI_Ssend
- MPI_Bsend
- MPI_Rsend

II. Resultados



III. Análise

Visto que esse programa roda localmente em uma única máquina, não envolveu o uso de uma rede externa que faça com que os processos se comuniquem. Isso gerou resultados interessantes onde foi possível transitar até 13Gb/s entre os dois processos. Isso não seria esperado caso fosse utilizado uma rede Gigabit por exemplo. A comunicação no último caso seria limitada pela própria capacidade de comunicação da infraestrutura de rede.

No mais, foi observado que o método MPI_Rsend foi o vencedor geral. Isso pode ser explicado pelo fato de que esse método assume que já existe alguém escutando a mensagem e também não aloca buffers intermediários. Acaba sendo um método mais perigoso mas também é mais eficiente.

IV. Anexo

```
#include <mpi.h>
#include <stdio.h>
```

```

#include <stdlib.h>
#include <string.h>

void executar_ping_pong(int rank, const char *metodo, int tamanho_maximo,
                        int iteracoes, FILE *arquivo_csv) {
    if (rank == 0) {
        printf("\n--- Testando %s ---\n", metodo);
        printf("%-15s %-20s %-20s\n", "Tamanho (B)", "Tempo Medio (s)",
            "Banda (MB/s)");
    }

    char *bsend_buffer = NULL;
    if (strcmp(metodo, "MPI_Bsend") == 0) {
        int tamanho_buffer = tamanho_maximo + MPI_BSEND_OVERHEAD;
        bsend_buffer = (char *)malloc(tamanho_buffer * sizeof(char));
        MPI_Buffer_attach(bsend_buffer, tamanho_buffer);
    }

    for (int tamanho = 8; tamanho <= tamanho_maximo; tamanho *= 4) {
        char *mensagem = (char *)malloc(tamanho * sizeof(char));
        memset(mensagem, 'x', tamanho);
        MPI_Status status;

        MPI_Barrier(MPI_COMM_WORLD);
        double tempo_inicio = MPI_Wtime();

        for (int i = 0; i < iteracoes; ++i) {
            if (rank == 0) {
                if (strcmp(metodo, "MPI_Send") == 0) {
                    MPI_Send(mensagem, tamanho, MPI_CHAR, 1, 0, MPI_COMM_WORLD);
                } else if (strcmp(metodo, "MPI_Ssend") == 0) {
                    MPI_Ssend(mensagem, tamanho, MPI_CHAR, 1, 0, MPI_COMM_WORLD);
                } else if (strcmp(metodo, "MPI_Bsend") == 0) {
                    MPI_Bsend(mensagem, tamanho, MPI_CHAR, 1, 0, MPI_COMM_WORLD);
                } else if (strcmp(metodo, "MPI_Rsend") == 0) {
                    MPI_Rsend(mensagem, tamanho, MPI_CHAR, 1, 0, MPI_COMM_WORLD);
                }

                MPI_Recv(mensagem, tamanho, MPI_CHAR, 1, 0, MPI_COMM_WORLD, &status);
            } else if (rank == 1) {
                MPI_Recv(mensagem, tamanho, MPI_CHAR, 0, 0, MPI_COMM_WORLD, &status);

                if (strcmp(metodo, "MPI_Send") == 0) {
                    MPI_Send(mensagem, tamanho, MPI_CHAR, 0, 0, MPI_COMM_WORLD);
                } else if (strcmp(metodo, "MPI_Ssend") == 0) {
                    MPI_Ssend(mensagem, tamanho, MPI_CHAR, 0, 0, MPI_COMM_WORLD);
                } else if (strcmp(metodo, "MPI_Bsend") == 0) {
                    MPI_Bsend(mensagem, tamanho, MPI_CHAR, 0, 0, MPI_COMM_WORLD);
                } else if (strcmp(metodo, "MPI_Rsend") == 0) {
                    MPI_Rsend(mensagem, tamanho, MPI_CHAR, 0, 0, MPI_COMM_WORLD);
                }
            }
        }
    }
}

```

```

double tempo_fim = MPI_Wtime();

if (rank == 0) {
    double tempo_medio = (tempo_fim - tempo_inicio) / (2.0 * iteracoes);
    double banda_mb = (tamanho / (1024.0 * 1024.0)) / tempo_medio;

    printf("%-15d %-20.8f %-20.2f\n", tamanho, tempo_medio, banda_mb);
    fprintf(arquivo_csv, "%s,%d,%.8f,%.2f\n", metodo, tamanho, tempo_medio,
            banda_mb);
}

free(mensagem);
}

if (strcmp(metodo, "MPI_Bsend") == 0) {
    int tamanho_desalocado;
    char *ponteiro_buffer;
    MPI_Buffer_detach(&ponteiro_buffer, &tamanho_desalocado);
    free(bsend_buffer);
}
}

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size != 2) {
        if (rank == 0) {
            fprintf(stderr, "Erro: Este programa deve ser executado com exatamente 2 "
                    "processos.\n");
        }

        MPI_Abort(MPI_COMM_WORLD, 1);
    }

    FILE *arquivo_csv = NULL;

    if (rank == 0) {
        arquivo_csv = fopen("resultados_mpi.csv", "w");
        if (arquivo_csv == NULL) {
            fprintf(stderr, "Erro ao criar o arquivo CSV.\n");
            MPI_Abort(MPI_COMM_WORLD, 1);
        }
        fprintf(arquivo_csv, "Metodo,Tamanho_Bytes,Tempo_Segundos,Banda_MB\n");
    }

    const int TAMANHO_MAXIMO = 1e8;
    const int ITERACOES = 100;

    executar_ping_pong(rank, "MPI_Send", TAMANHO_MAXIMO, ITERACOES, arquivo_csv);
    executar_ping_pong(rank, "MPI_Ssend", TAMANHO_MAXIMO, ITERACOES, arquivo_csv);
    executar_ping_pong(rank, "MPI_Bsend", TAMANHO_MAXIMO, ITERACOES, arquivo_csv);

```

```
executar_ping_pong(rank, "MPI_Rsend", TAMANHO_MAXIMO, ITERACOES, arquivo_csv);  
  
MPI_Finalize();  
return 0;  
}
```